

Kapitel 1.3

Zusammenhang

Definition: Sei $G = (V, E)$ ein Graph.

G heisst **zusammenhängend**, wenn

$\forall u, v \in V, u \neq v$ gilt: es gibt einen u-v-Pfad in G.

Heute:

Gegeben ein zusammenhängender Graph.

Wie (sehr) zusammenhängend ist dieser Graph?

Definition: Sei $G = (V, E)$ ein Graph.

G heisst **k-zusammenhängend**, wenn gilt:

- $|V| \geq k + 1$ und
- $\forall X \subseteq V$ mit $|X| < k$ ist $G[V \setminus X]$ zusammenhängend

Definition: Sei $G = (V, E)$ ein Graph.

G heisst **k-kanten-zusammenhängend**, wenn gilt:

- $\forall X \subseteq E$ mit $|X| < k$ ist $(V, E \setminus X)$ zusammenhängend

Satz von Menger:

Sei $G = (V, E)$ ein Graph. Dann gilt:

G ist **k-zusammenhängend** genau dann wenn (gdw.)

$\forall u, v \in V, u \neq v$ gibt es **k intern-knotendisjunkte** u-v-Pfade

Satz von Menger:

Sei $G = (V, E)$ ein Graph. Dann gilt:

G ist **k-kanten-zusammenhängend** genau dann wenn (gdw.)

$\forall u, v \in V, u \neq v$ gibt es **k kantendisjunkte** u-v-Pfade

Satz von Menger:

Sei $G = (V, E)$ ein Graph. Dann gilt:

G ist **k-kanten-zusammenhängend** genau dann wenn (gdw.)

$\forall u, v \in V, u \neq v$ gibt es **k kantendisjunkte** u-v-Pfade

Beweisidee:

Wir zeigen: $\forall u, v \in V, u \neq v$ gilt

maximale Anzahl intern-kantendisjunkter u-v Pfade

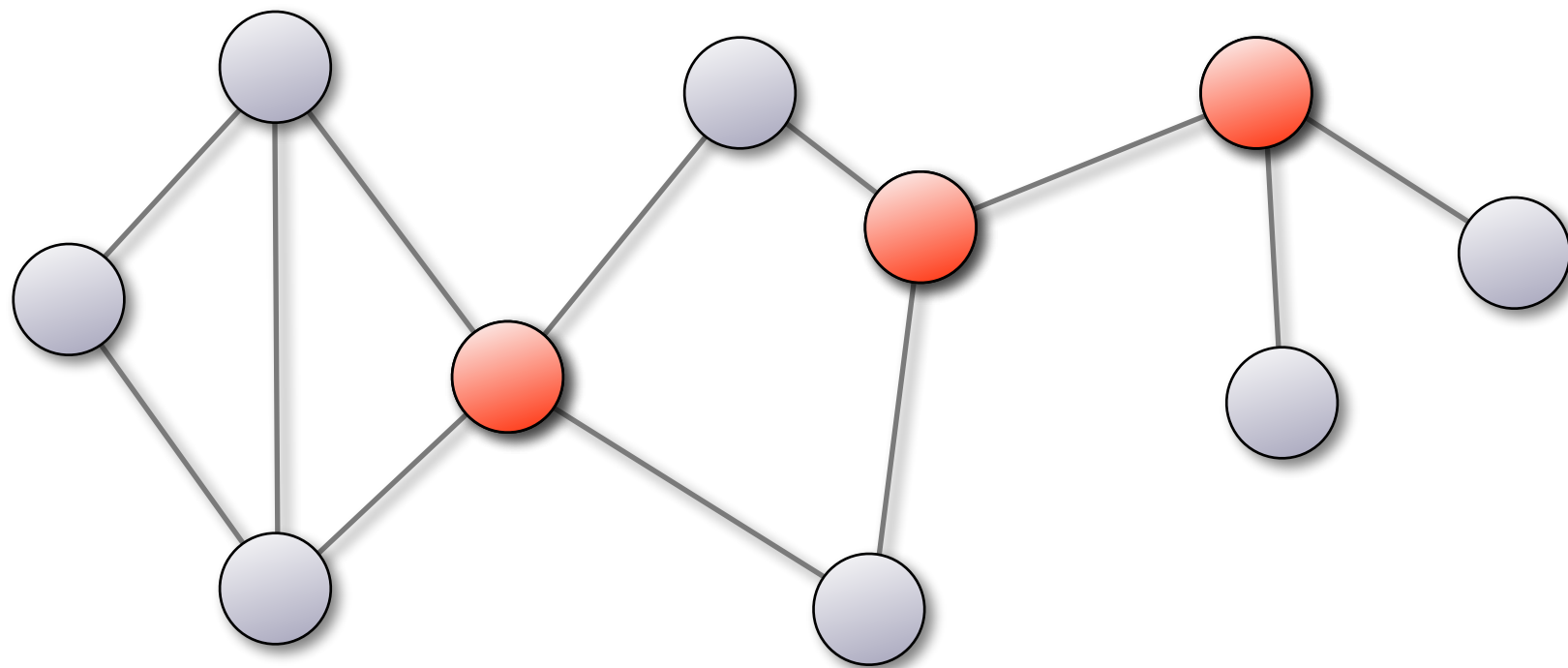
= **minimale** Anzahl Kanten, die man entfernen muss um u und v zu trennen (d.h. liegen nicht mehr in der gleichen Zshgskomp.)

Bew: „ $\leq$ “: klar. (wir müssen von jedem Pfad mind. eine Kante entfernen) „ $\geq$ “: schwieriger...

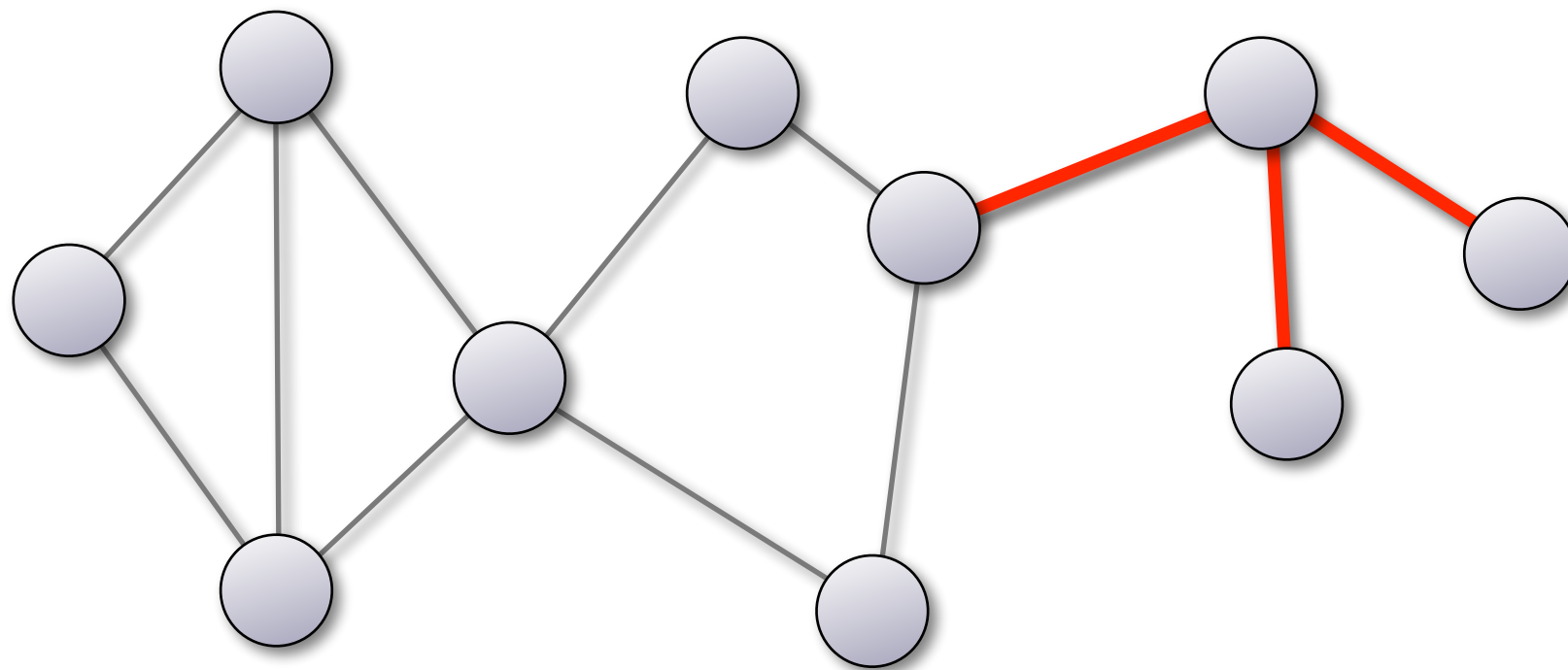
Heute:

Spezialfall $k=1$

Definition: Sei $G = (V, E)$ ein zusammenhängender Graph.
Ein Knoten $v \in V$ heisst *Artikulationsknoten* (engl. cut vertex)
gdw. $G[V \setminus \{v\}]$ nicht zusammenhängend ist



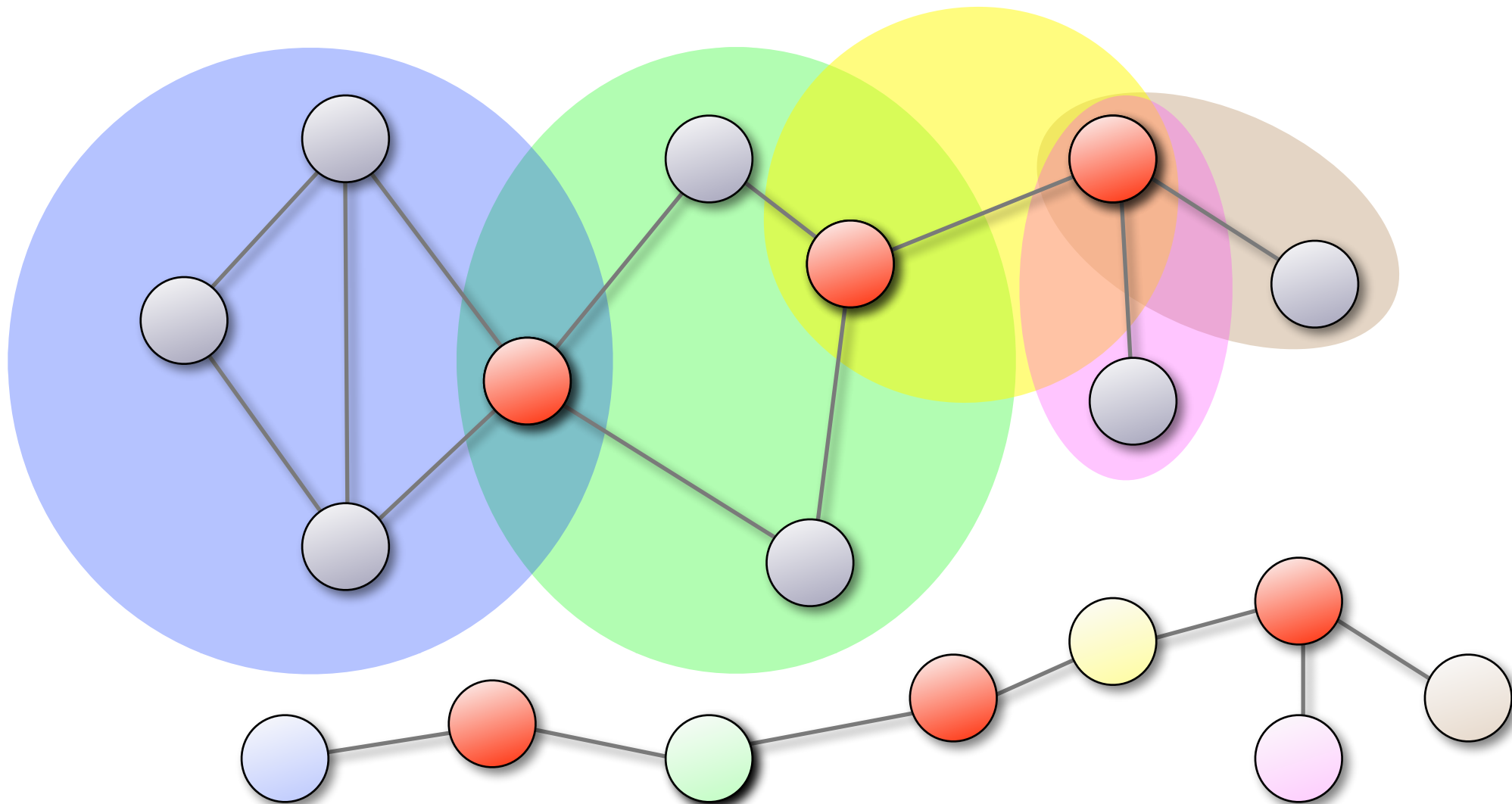
Definition: Sei $G = (V, E)$ ein zusammenhängender Graph.
Ein Kante $e \in E$ heisst **Brücke** (engl. cut edge)
gdw. $G - e$ nicht zusammenhängend ist



Definition: Sei $G = (V, E)$ ein zusammenhängender Graph.

Der **Block-Graph** von G ist der bipartite Graph $T = (A \uplus B, E_T)$ mit

- $A = \{\text{Artikulationsknoten von } G\}$.
- $B = \{\text{Blöcke von } G\}$.
- $\forall a \in A, b \in B : \{a, b\} \in E_T \iff a \text{ inzident zu einer Kante in } b$.



Satz: Ist G zusammenhängend, so ist der Blockgraph von G ein Baum.

Wie findet man Artikulationsknoten
bzw Brücken effizient ?

Exkurs

(Wdh aus dem letzten Semester)

BFS-VISIT-ITERATIVE(G, v)

```
1  $Q \leftarrow \emptyset$                                 ▷ Initialisiere  $Q$ 
2 Markiere  $v$  als aktiv
3 ENQUEUE( $Q, v$ )                                ▷ Füge  $v$  zu  $Q$ 
4 while  $Q \neq \emptyset$  do
5      $w \leftarrow \text{DEQUEUE}(Q)$                 ▷ Aktueller Knoten
6     Markiere  $w$  als besucht
7     for each  $(w, x) \in E$  do
8         if  $x$  nicht aktiv und  $x$  noch nicht besucht then
9             Markiere  $x$  als aktiv
10            ENQUEUE( $Q, x$ )
```

1

DFS-VISIT-ITERATIVE(G, v)

```
1  $S \leftarrow \emptyset$ 
2 PUSH( $S, v$ )
3 while  $S \neq \emptyset$  do
4      $w \leftarrow \text{POP}(S)$ 
5     if  $w$  noch nicht besucht then
6         Markiere  $w$  als besucht
7         for each  $(w, x) \in E$  in reverse order do
8             if  $x$  noch nicht besucht then
9                 PUSH( $S, x$ )
```

BFS-VISIT-ITERATIVE(G, v)

```
1  $Q \leftarrow \emptyset$                                  $\triangleright$  Initialisiere  $Q$ 
2 Markiere  $v$  als aktiv
3 ENQUEUE( $Q, v$ )                                 $\triangleright$  Füge  $v$  zu  $Q$ 
4 while  $Q \neq \emptyset$  do
5      $w \leftarrow$  DEQUEUE( $Q$ )                     $\triangleright$  Aktueller Knoten
6     Markiere  $w$  als besucht
7     for each  $(w, x) \in E$  do
8         if  $x$  nicht aktiv und  $x$  noch nicht besucht then
9             Markiere  $x$  als aktiv
10            ENQUEUE( $Q, x$ )
```



Queue:
First-in-first-out

DFS-VISIT-ITERATIVE(G, v)

```
1  $S \leftarrow \emptyset$ 
2 PUSH( $S, v$ )
3 while  $S \neq \emptyset$  do
4      $w \leftarrow$  POP( $S$ )
5     if  $w$  noch nicht besucht then
6         Markiere  $w$  als besucht
7         for each  $(w, x) \in E$  in reverse order do
8             if  $x$  noch nicht besucht then
9                 PUSH( $S, x$ )
```

Stack:
Last-in-first-out



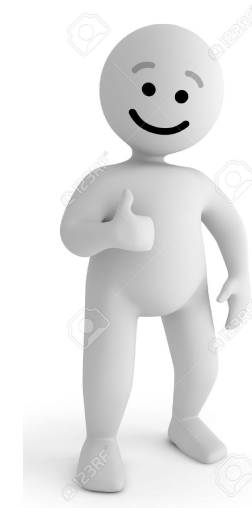
BFS-VISIT-ITERATIVE(G, v)

```
1  $Q \leftarrow \emptyset$ 
2 Markiere  $v$  als aktiv
3 ENQUEUE( $Q, v$ )
4 while  $Q \neq \emptyset$  do
5      $w \leftarrow$  DEQUEUE( $Q$ )
6     Markiere  $w$  als besucht
7     for each  $(w, x) \in E$  do
8         if  $x$  nicht aktiv und  $x$  noch nicht besucht then
9             Markiere  $x$  als aktiv
10            ENQUEUE( $Q, x$ )
```

▷ Initialisiere

▷ Füge v zu

▷ Aktueller



kürzeste Pfade

1

DFS-VISIT-ITERATIVE(G, v)

```
1  $S \leftarrow \emptyset$ 
2 PUSH( $S, v$ )
3 while  $S \neq \emptyset$  do
4      $w \leftarrow$  POP( $S$ )
5     if  $w$  noch nicht besucht then
6         Markiere  $w$  als besucht
7         for each  $(w, x) \in E$  in reverse order do
8             if  $x$  noch nicht besucht then
9                 PUSH( $S, x$ )
```



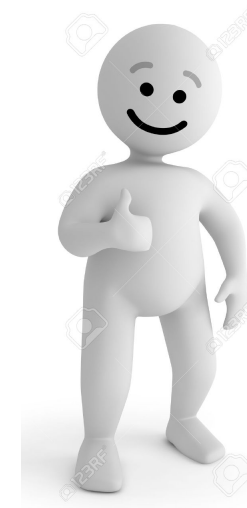
BFS-VISIT-ITERATIVE(G, v)

```
1  $Q \leftarrow \emptyset$ 
2 Markiere  $v$  als aktiv
3 ENQUEUE( $Q, v$ )
4 while  $Q \neq \emptyset$  do
5      $w \leftarrow$  DEQUEUE( $Q$ )
6     Markiere  $w$  als besucht
7     for each  $(w, x) \in E$  do
8         if  $x$  nicht aktiv und  $x$  noch nicht besucht then
9             Markiere  $x$  als aktiv
10            ENQUEUE( $Q, x$ )
```

▷ Initialisiere

▷ Füge v zu

▷ Aktueller

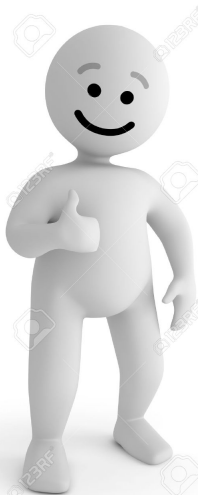


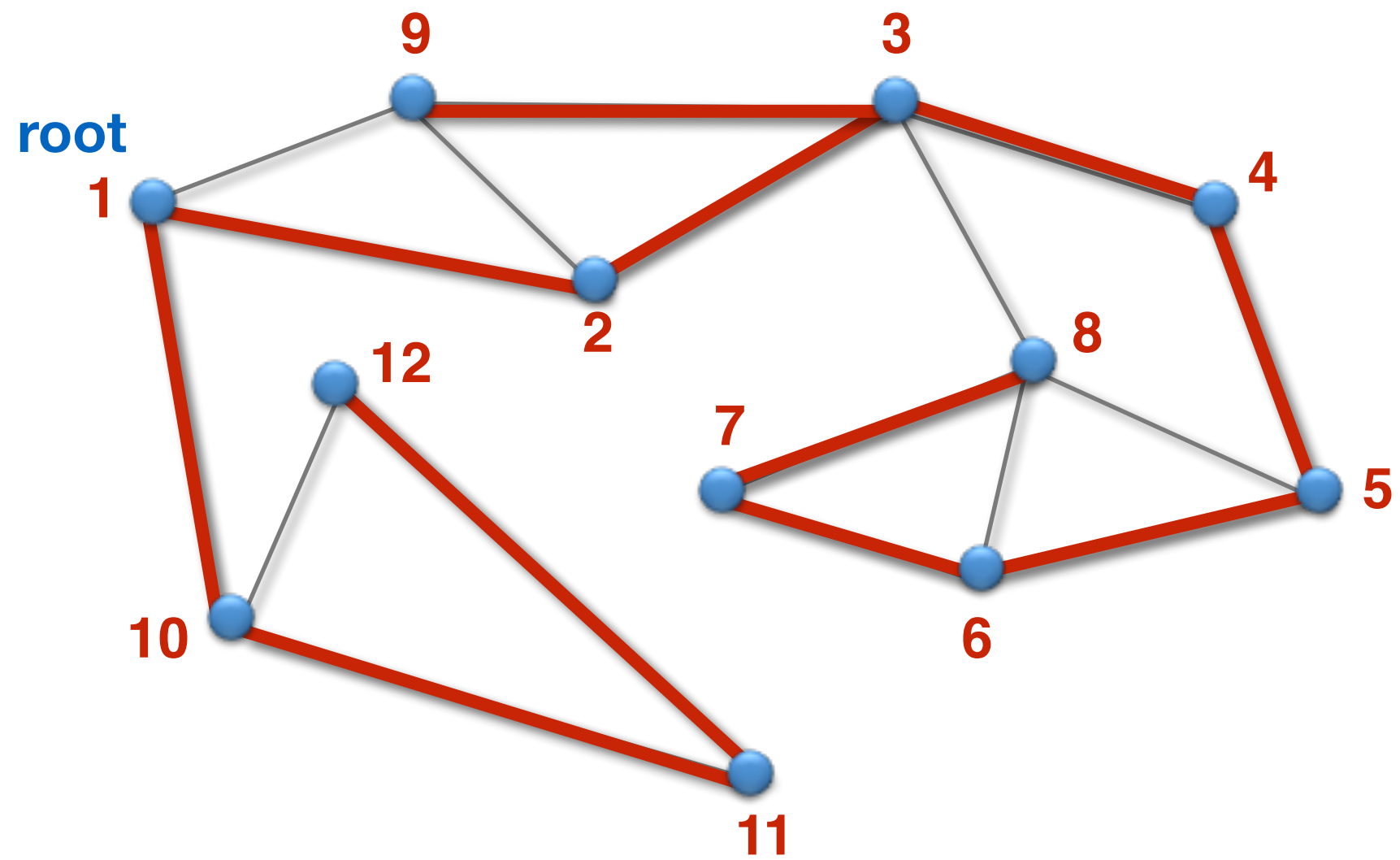
kürzeste Pfade

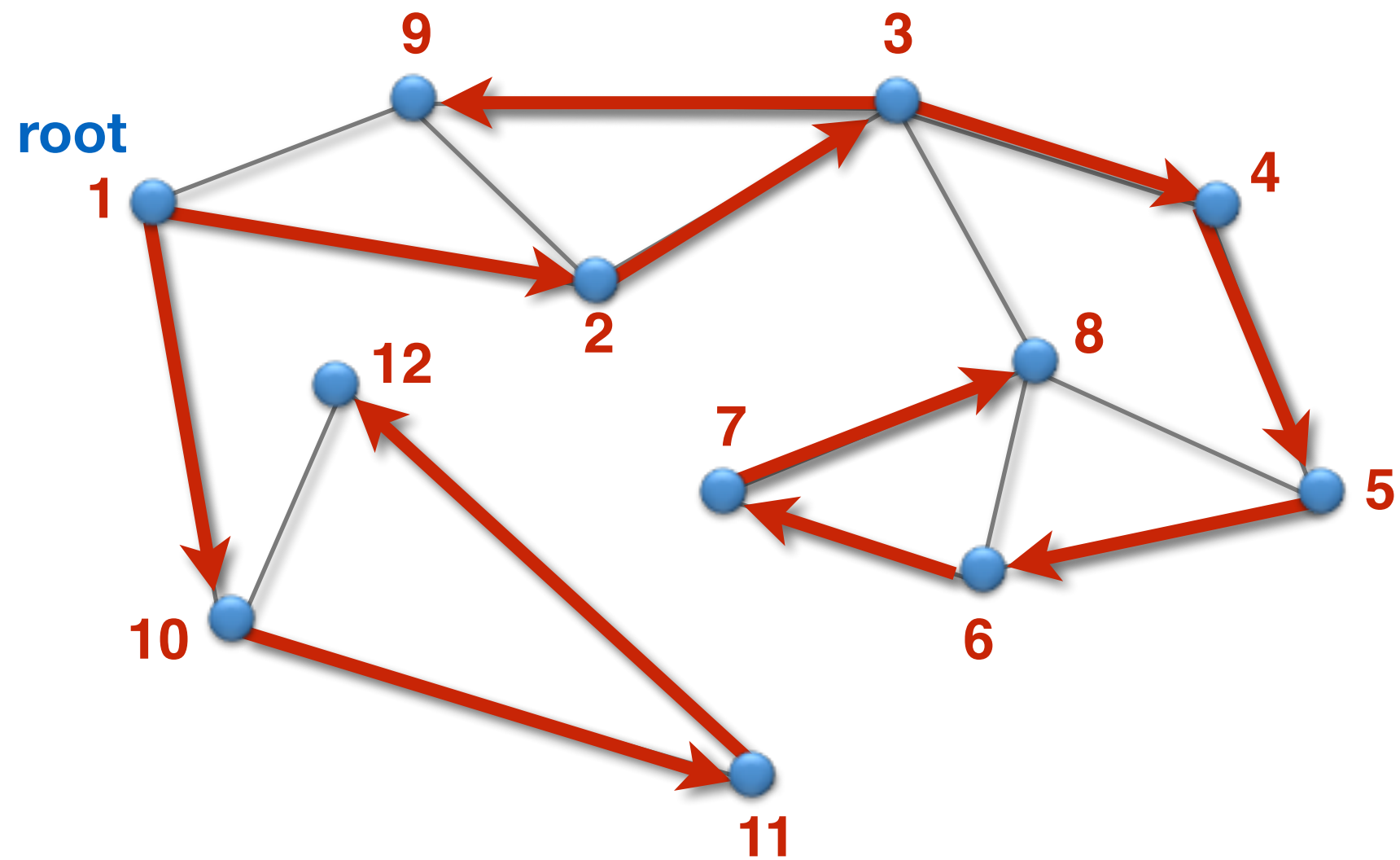
DFS-VISIT-ITERATIVE(G, v)

```
1  $S \leftarrow \emptyset$ 
2 PUSH( $S, v$ )
3 while  $S \neq \emptyset$  do
4      $w \leftarrow$  POP( $S$ )
5     if  $w$  noch nicht besucht then
6         Markiere  $w$  als besucht
7         for each  $(w, x) \in E$  in reverse order do
8             if  $x$  noch nicht besucht then
9                 PUSH( $S, x$ )
```

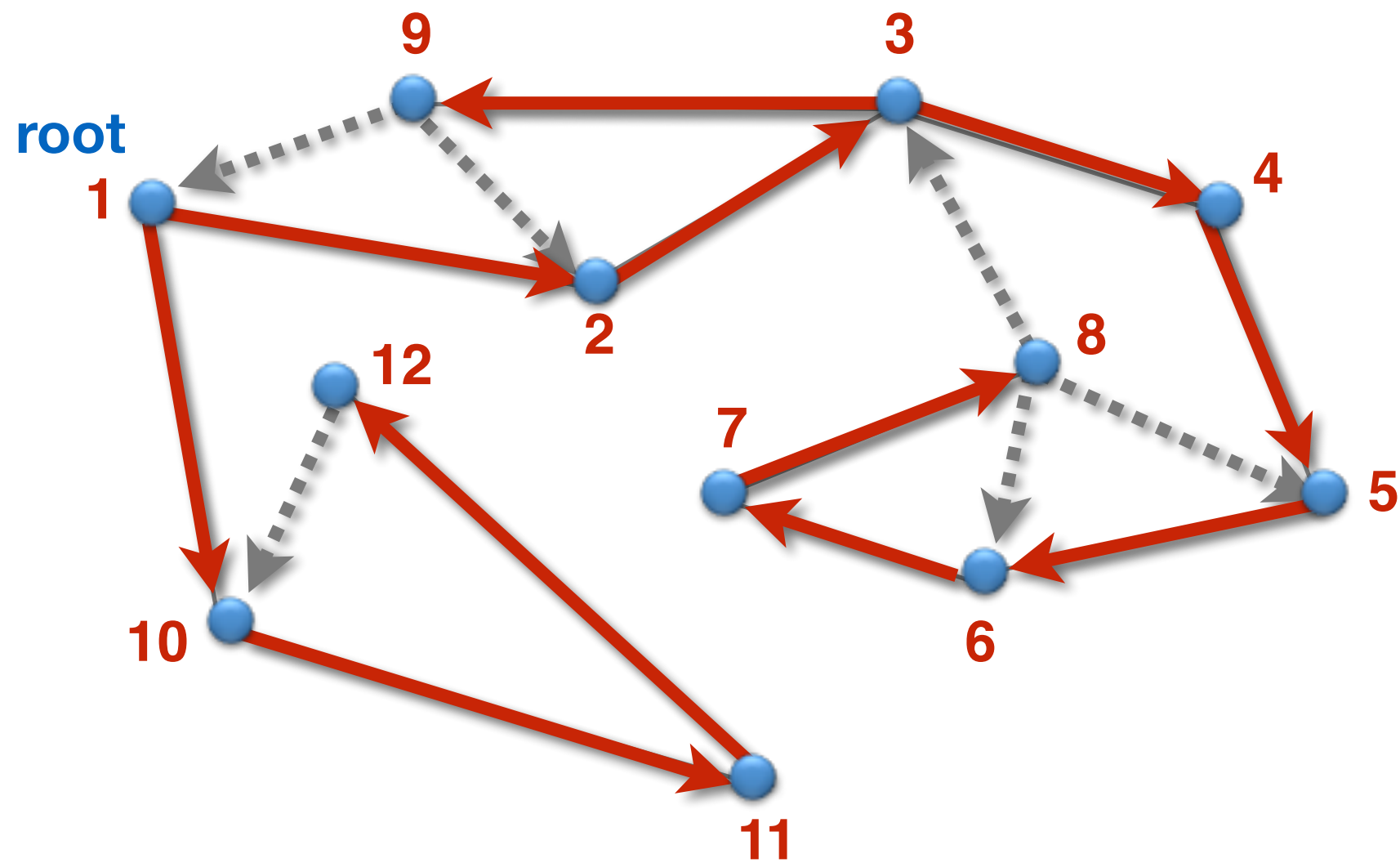
Artikulationsknoten

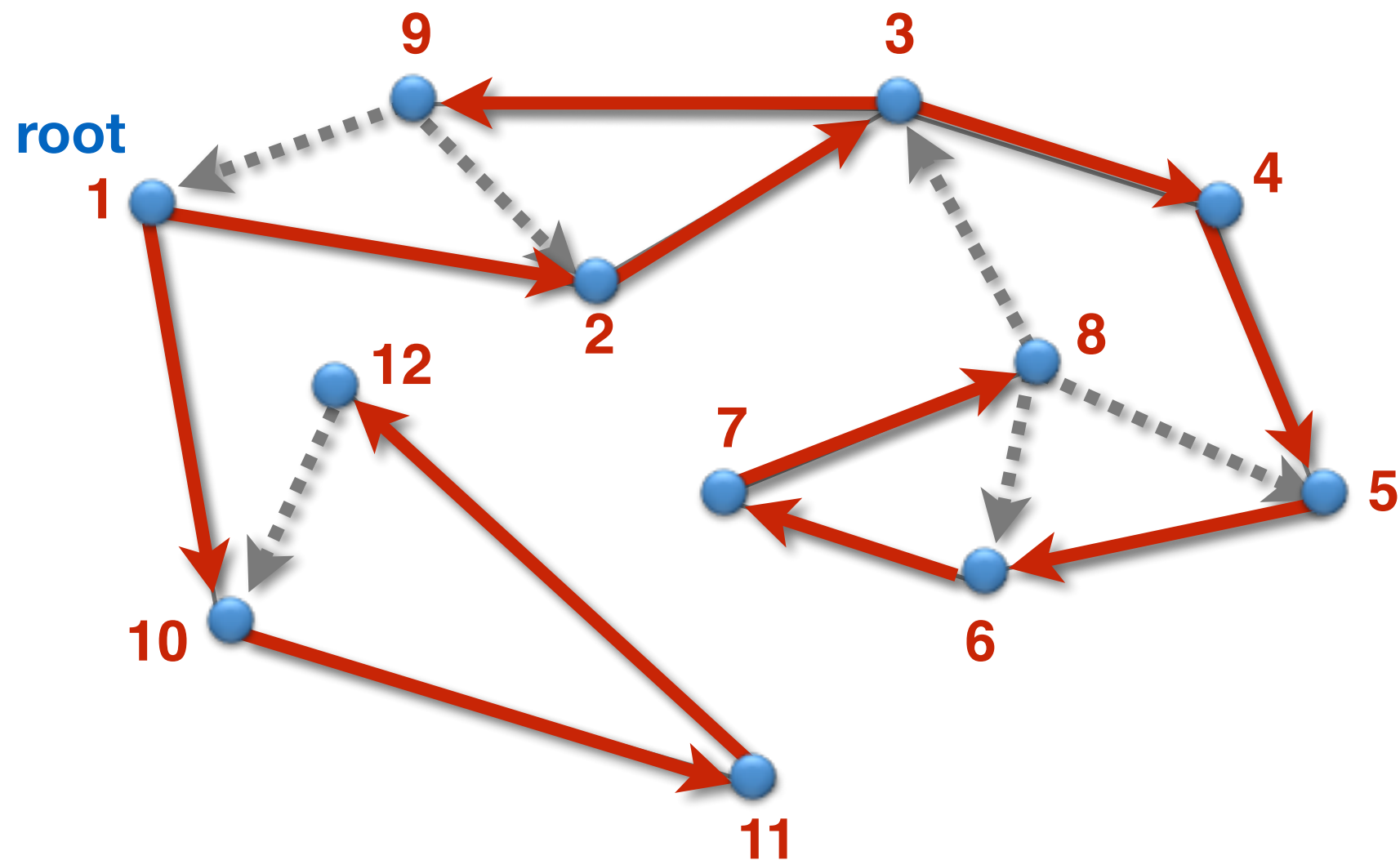






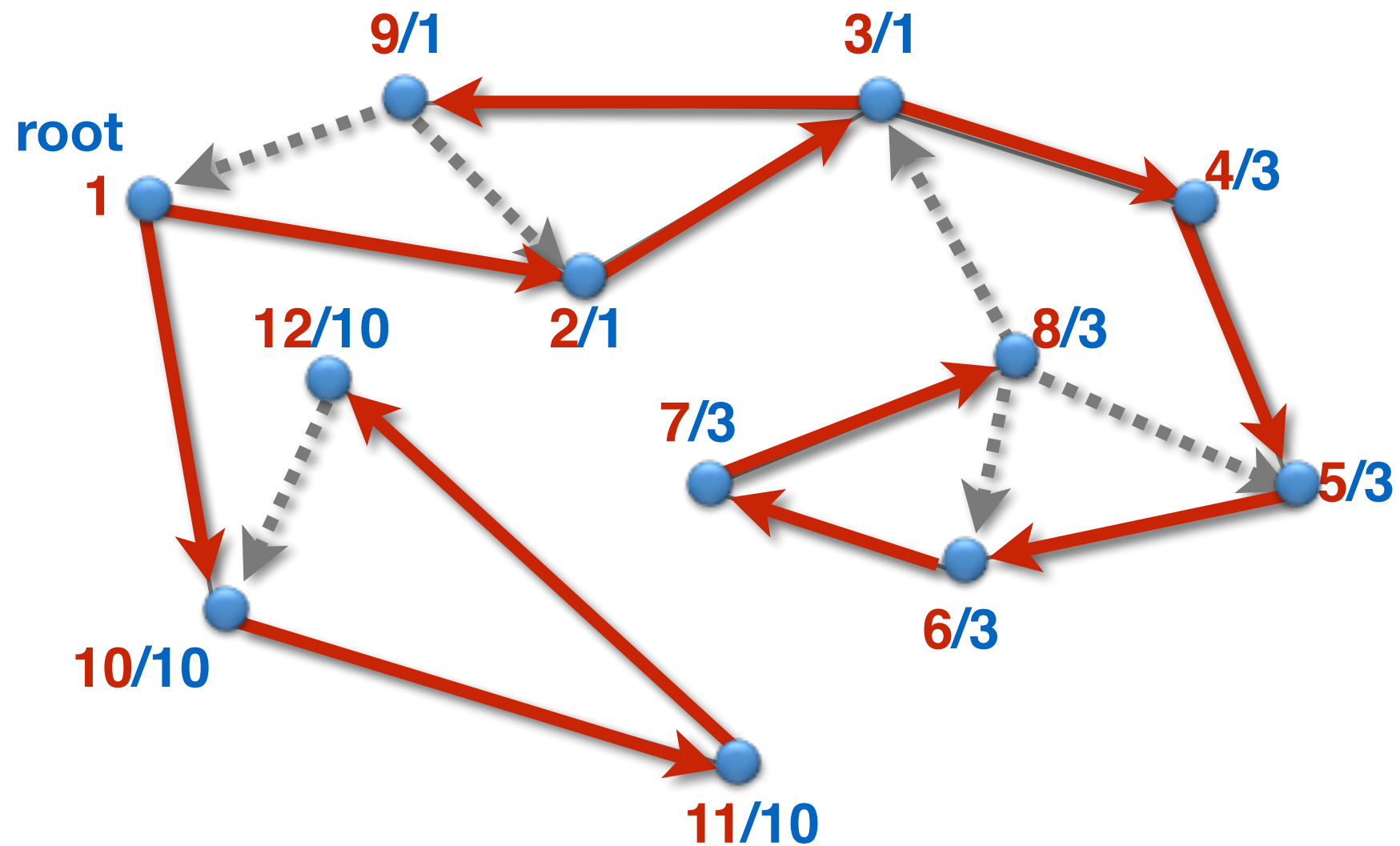
← Baumkanten
← Restkanten



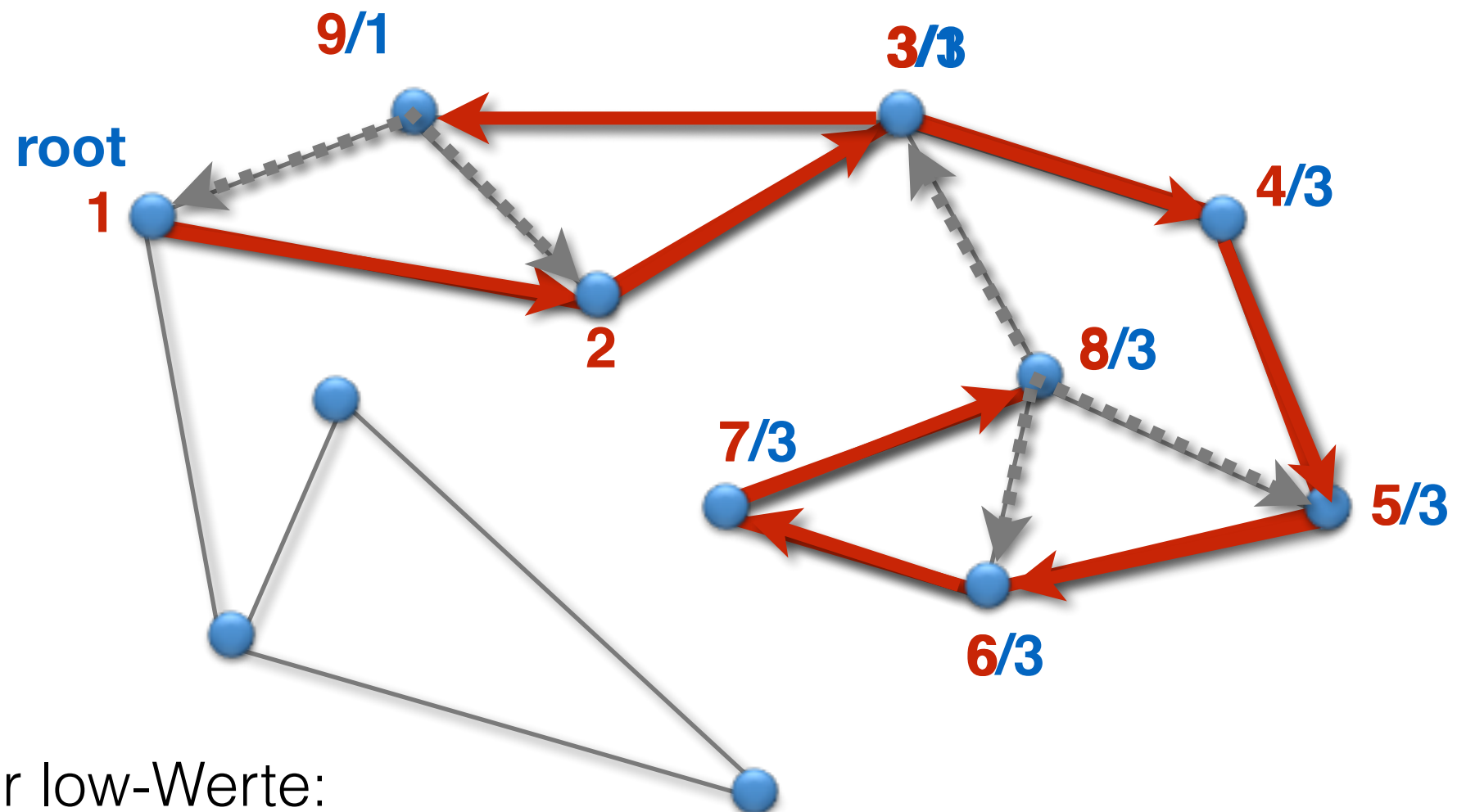


$\text{low}[v] :=$ kleinste dfs-Nummer, die man von v aus durch einen gerichteten Pfad aus (beliebig vielen) Baumkanten und maximal einer Restkante erreichen kann.

dfs / low



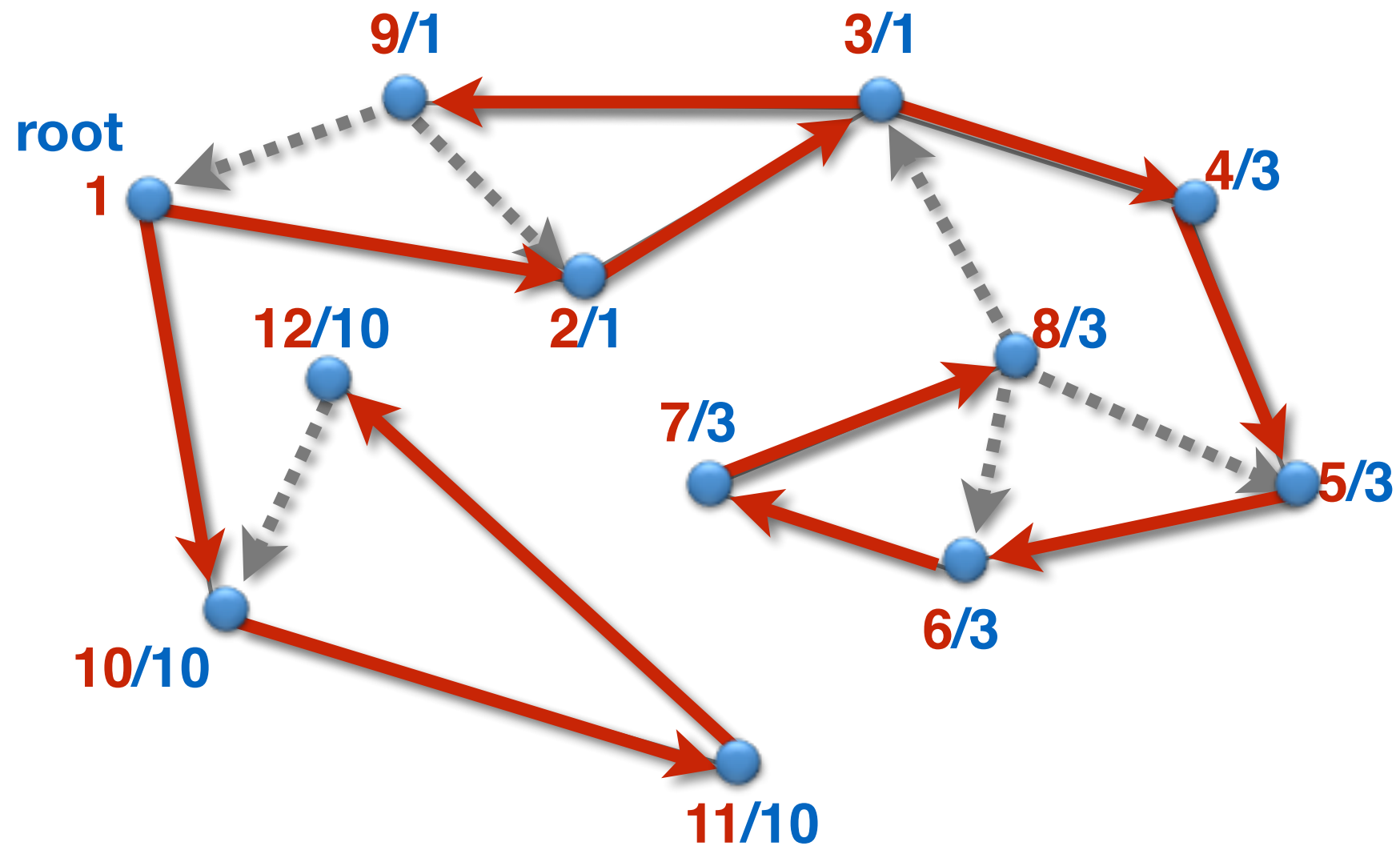
$\text{low}[v] :=$ kleinste dfs-Nummer, die man von v aus durch einen gerichteten Pfad aus (beliebig vielen) Baumkanten und maximal einer Restkante erreichen kann.



Berechnung der low-Werte:

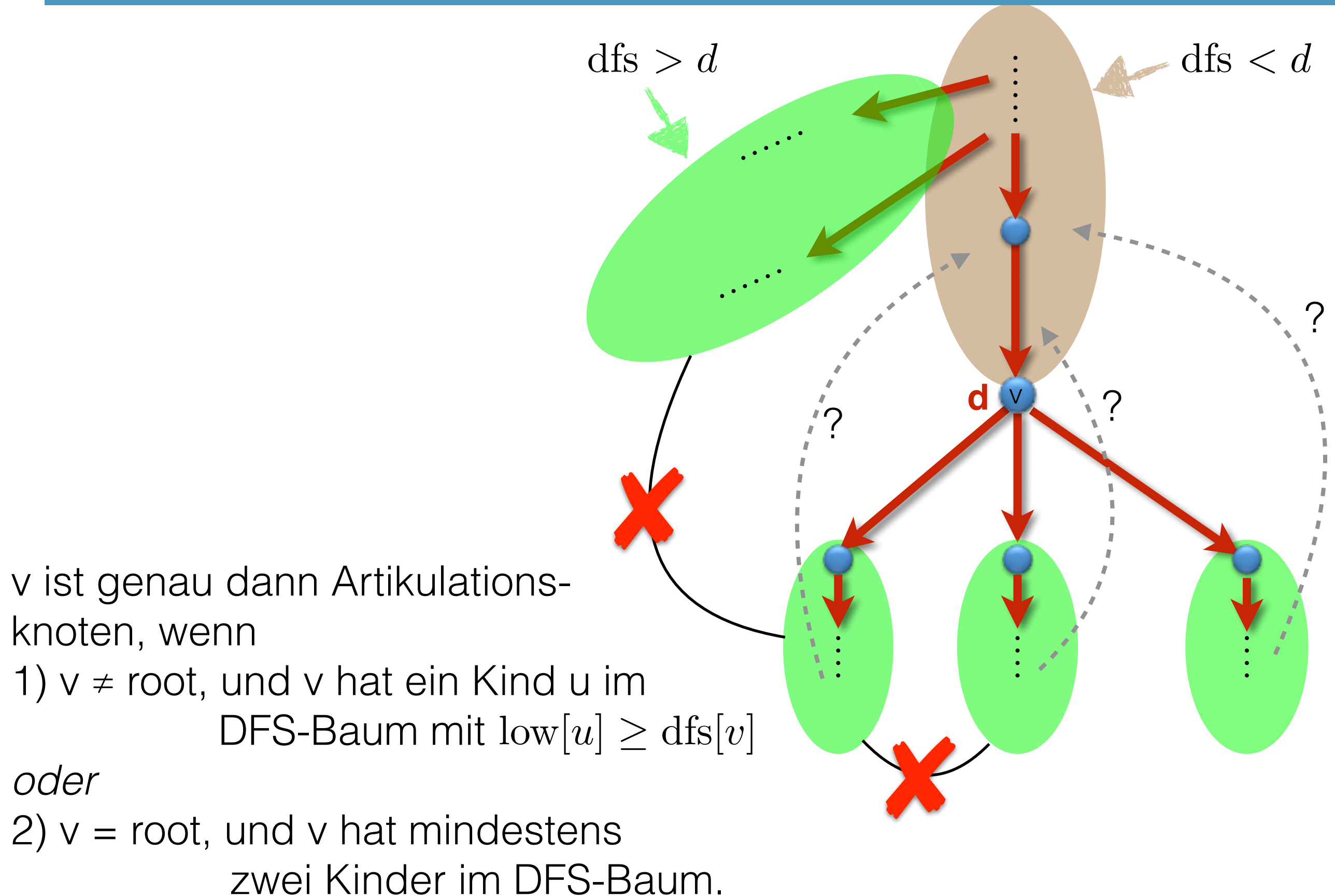
- Initialisierung mit dfs-Wert
- ggf update, wenn Restkanten gefunden werden
- ggf update, wenn Algorithmus während des backtracks zum Knoten zurückkehrt

dfs / low

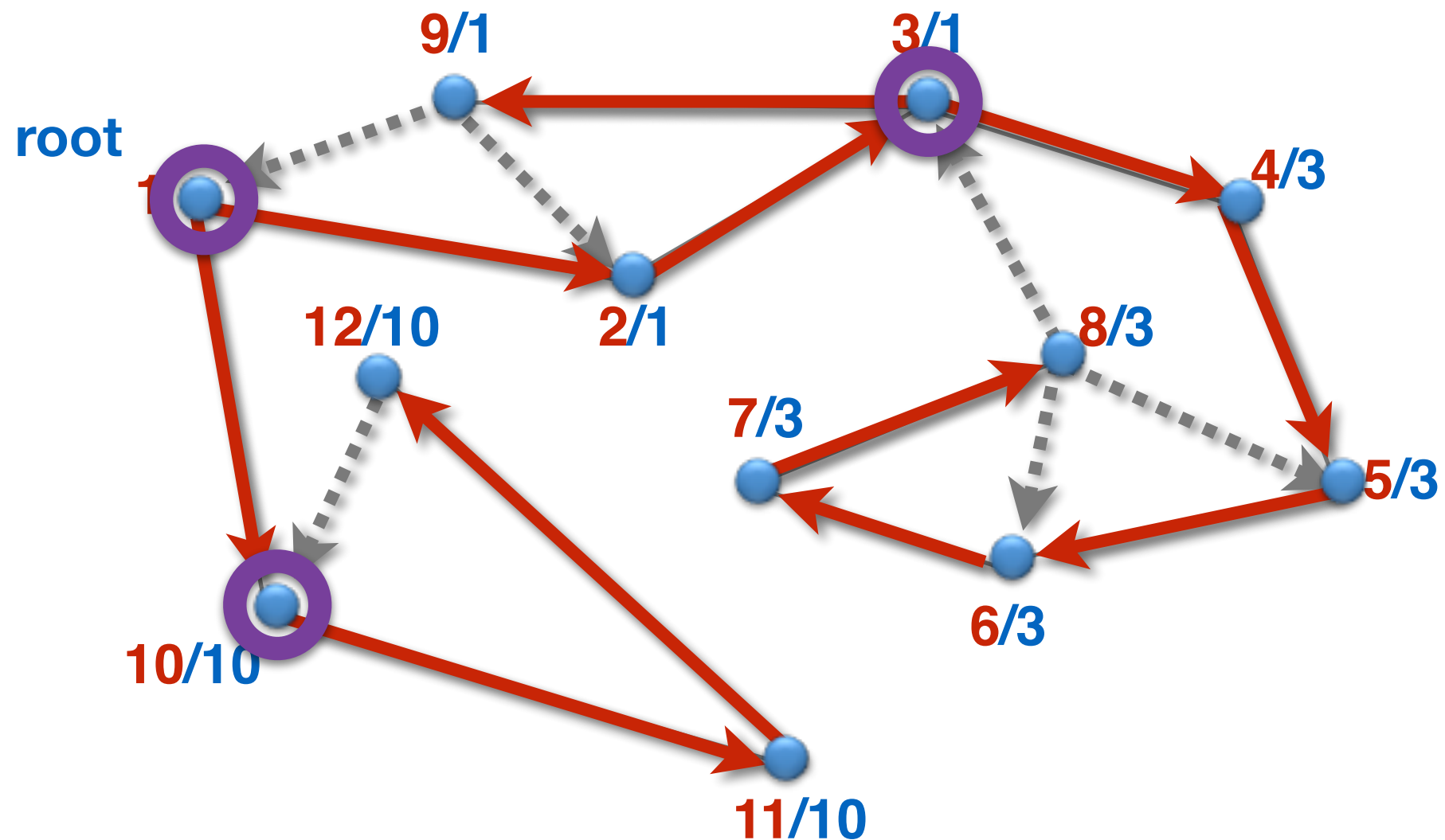


Alle low-Werte sind in Zeit $O(|V|+|E|)$ berechenbar:

$$\text{low}[v] = \min \left(\text{dfs}[v], \min_{(v,w) \in E} \begin{cases} \text{dfs}[w], & \text{falls } (v,w) \text{ Restkante} \\ \text{low}[w], & \text{falls } (v,w) \text{ Baumkante} \end{cases} \right)$$



dfs / low

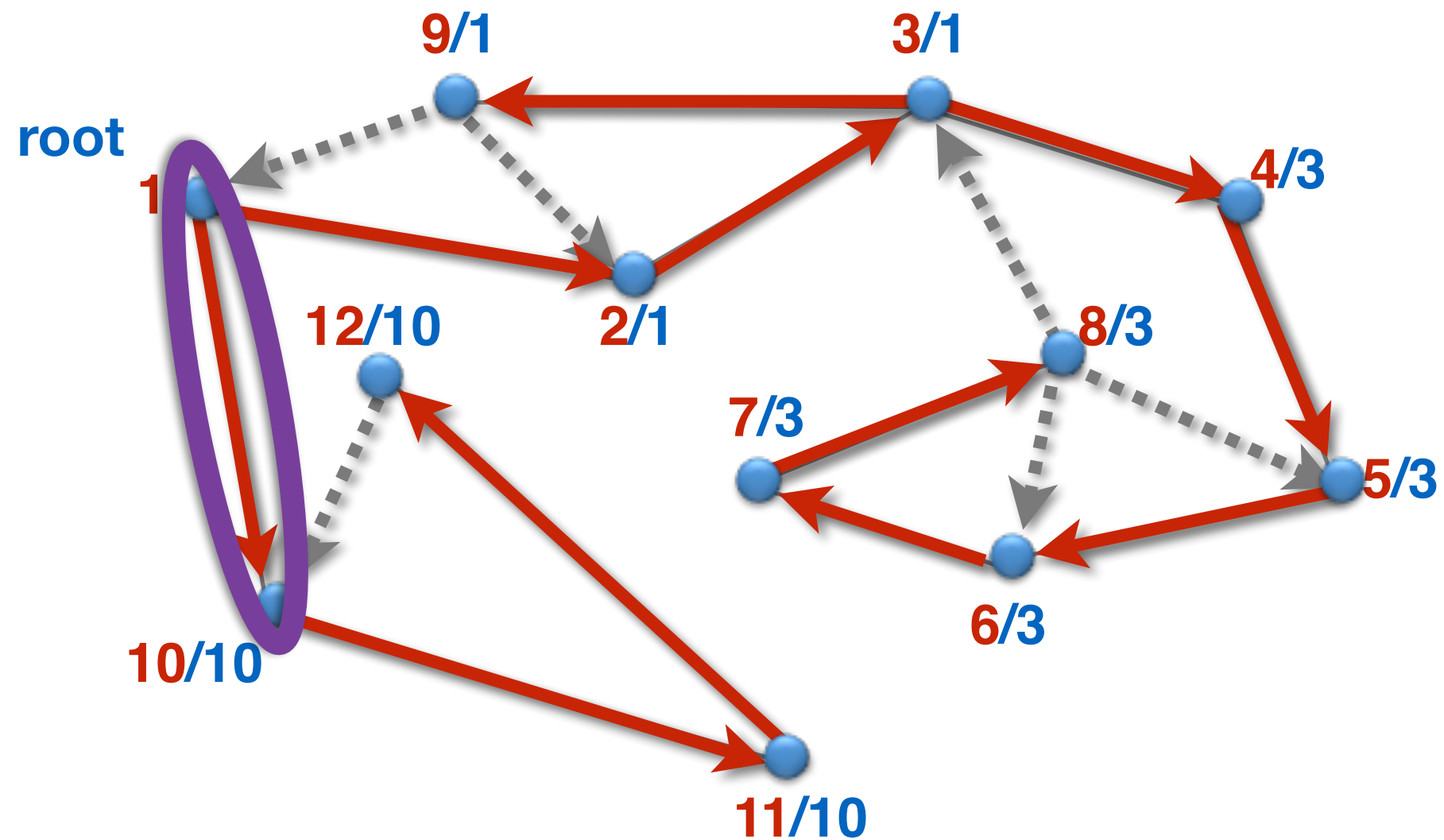


Ein Knoten v ist genau dann ein Artikulationsknoten, wenn

1) $v \neq \text{root}$, und v hat ein Kind u im DFS-Baum mit $\text{low}[u] \geq \text{dfs}[v]$,
oder

2) $v = \text{root}$, und v hat mindestens zwei Kinder im DFS-Baum.

dfs / low



Eine Baumkante $e = (v, w)$ (v Elternknoten, w Kindknoten) ist genau dann eine Brücke, wenn $\text{low}[w] > \text{dfs}[v]$

Restkanten sind niemals Brücken.

Satz: Die um die Berechnung von $\text{low}[]$ ergänzte **Tiefensuche** berechnet in einem zusammenhängenden Graphen alle Artikulationsknoten und Brücken in Zeit $O(m)$.

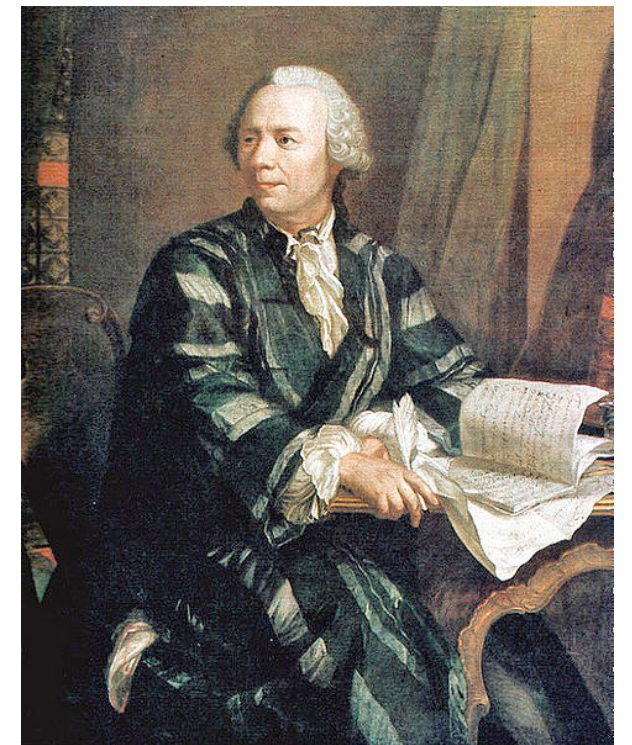
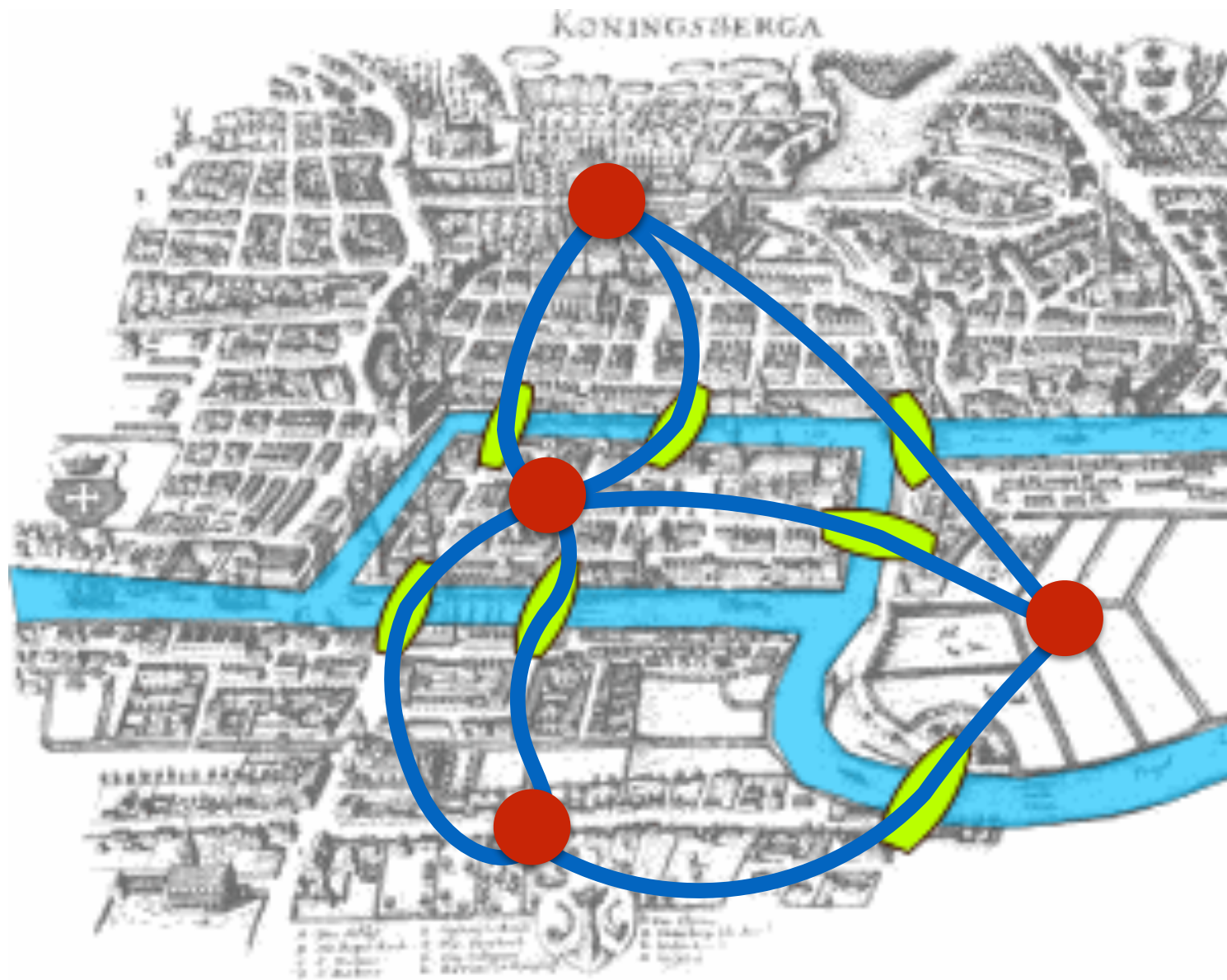
Kapitel 1.3

Kreise

Hamiltonkreise und Eulertouren

- Sei $G = (V, E)$ ein Graph.
- **Hamiltonkreis:**
 - Ein Kreis in G , der jeden **Knoten** genau einmal enthält.
- **Eulertour:**
 - Ein geschlossener Weg in G , der jede **Kante** genau einmal enthält.

Königsberger Brückenproblem



Leonard Euler
(1707 - 1783)

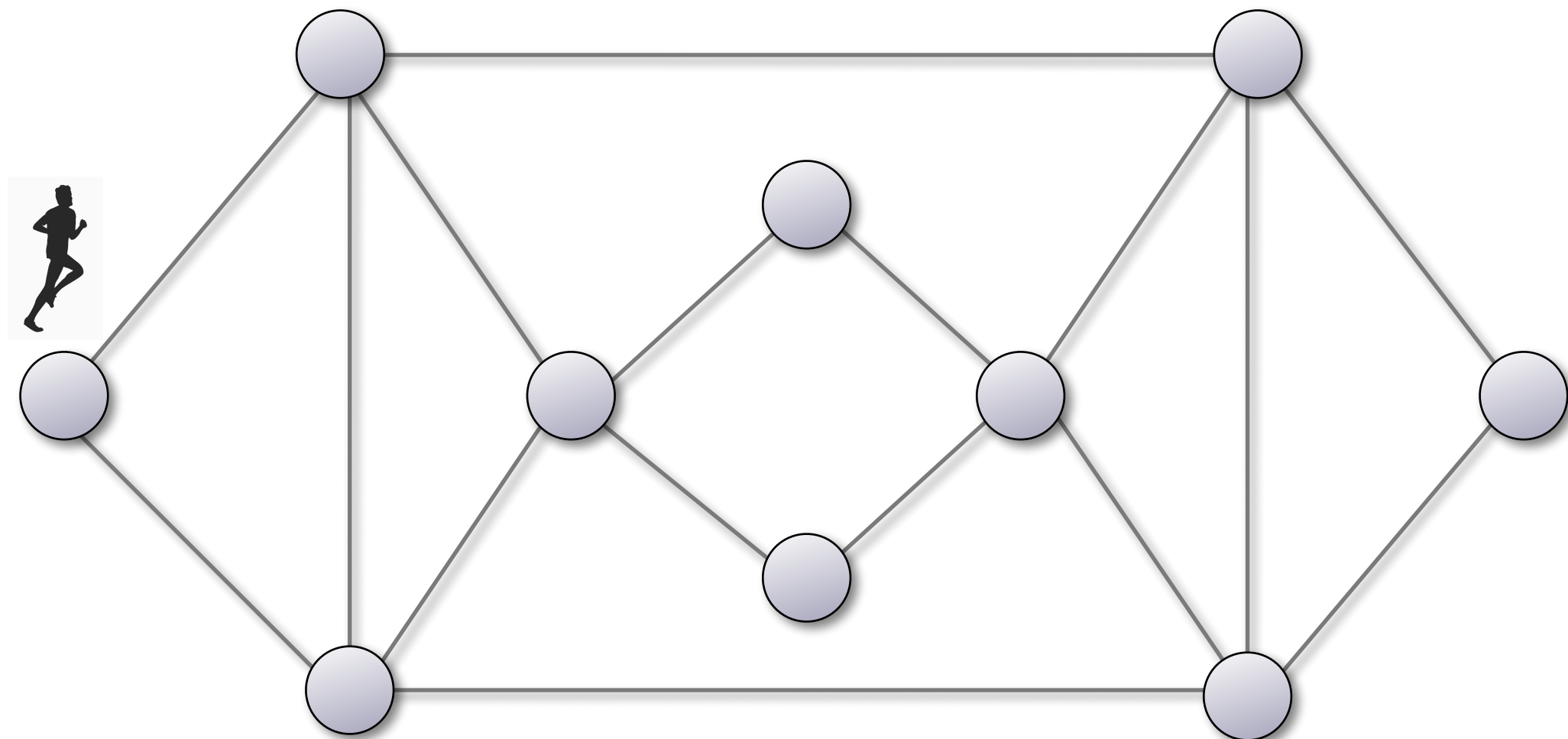
Gesucht: Eulertour

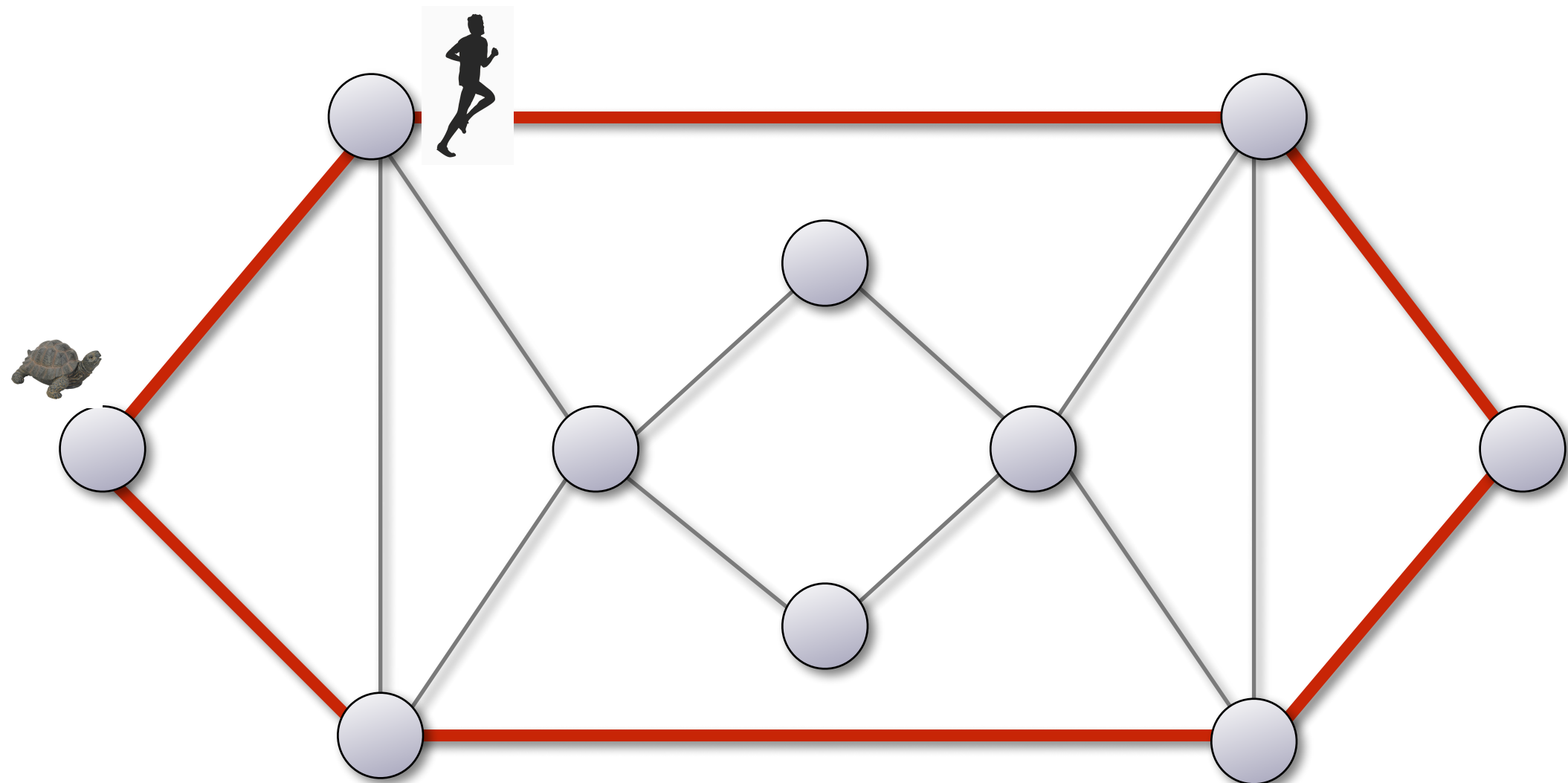
Eulertour: Charakterisierung

Satz: Ein zusammenhängender Graph $G = (V, E)$ enthält eine Eulertour

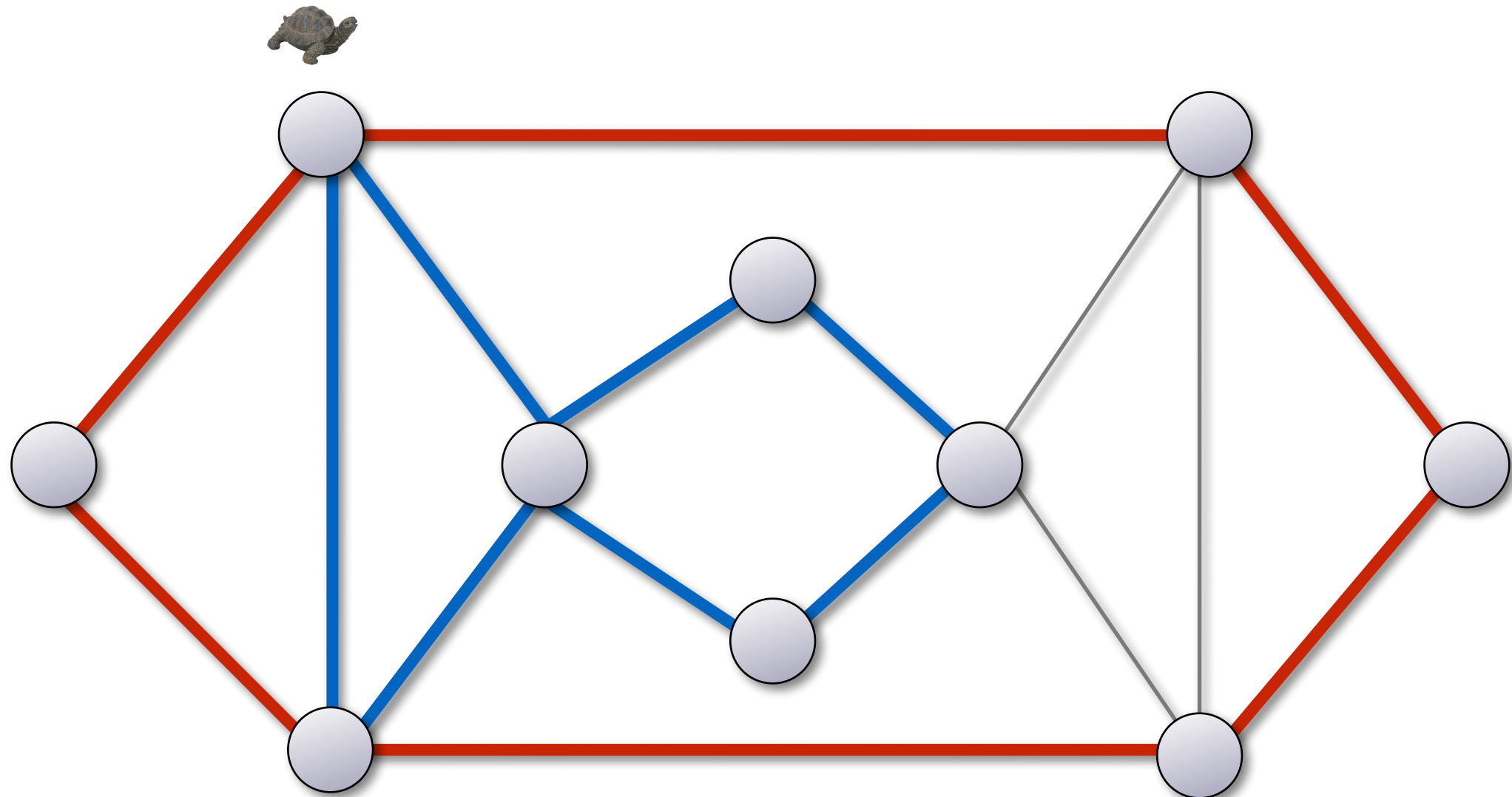
gdw. der Grad jedes Knotens gerade ist.

... und eine solche kann man in $O(|E|)$ Zeit finden

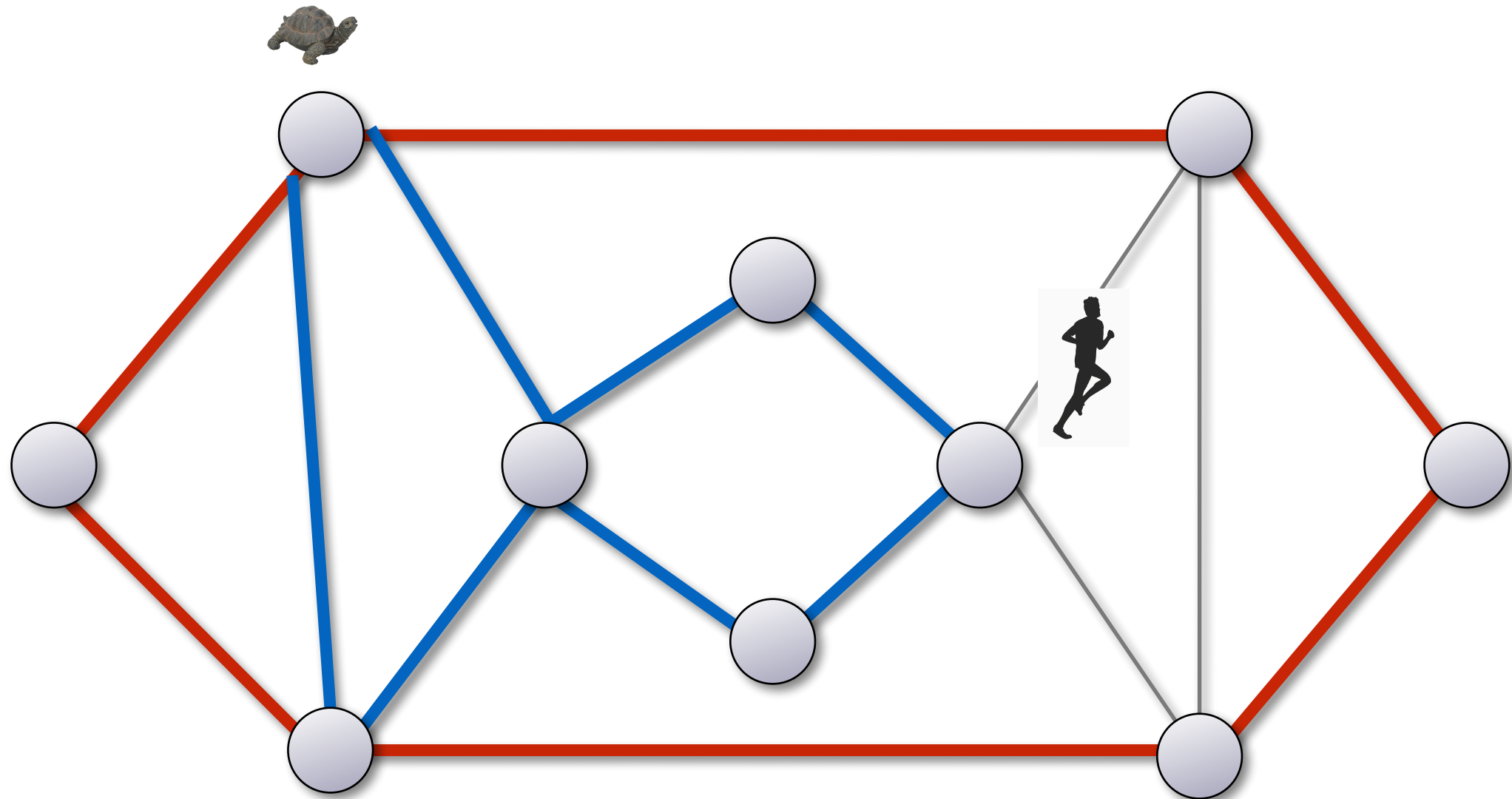




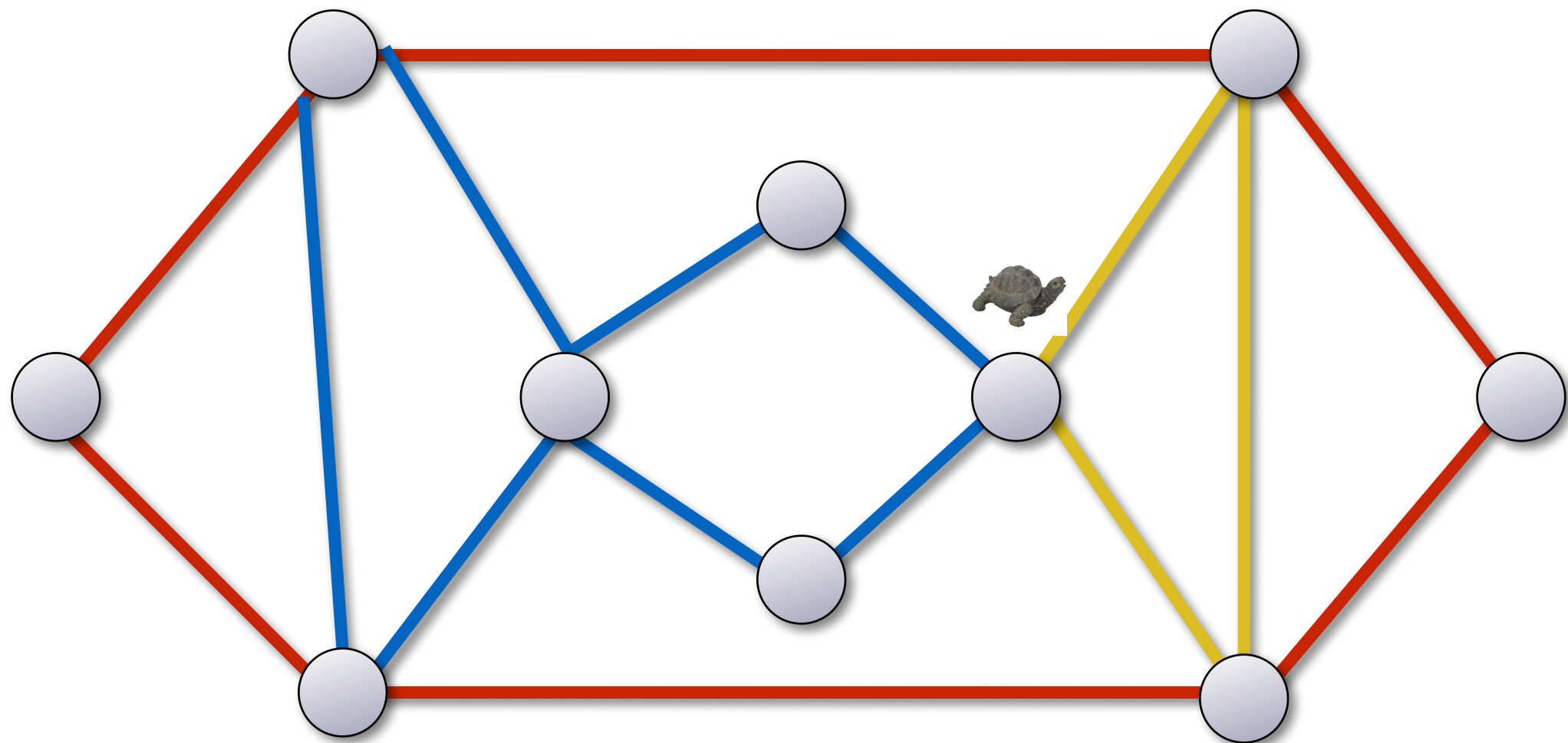
Idee des Algorithmus



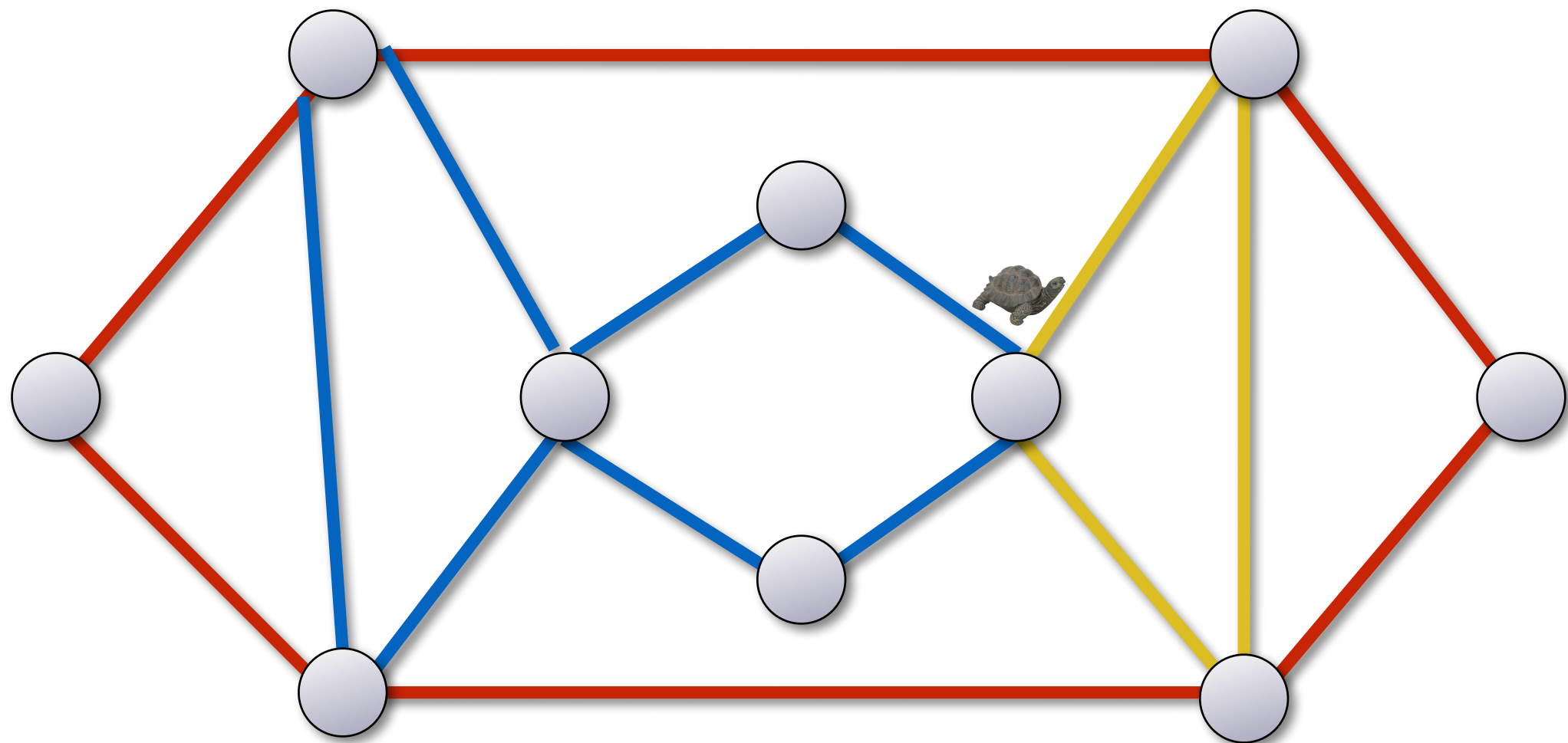
Idee des Algorithmus



Idee des Algorithmus

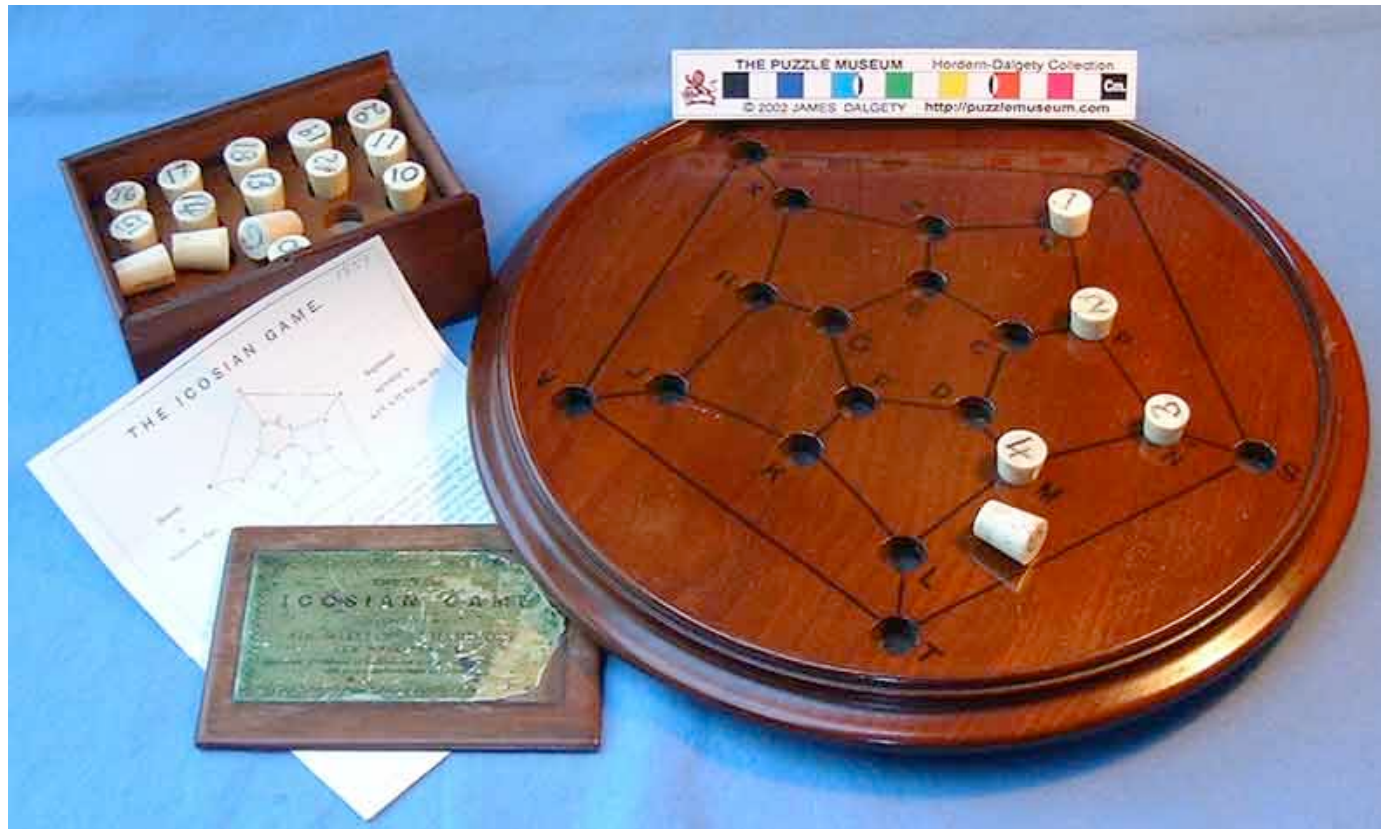


Idee des Algorithmus



Hamiltonkreise und Eulertouren

- Sei $G = (V, E)$ ein Graph.
- **Hamiltonkreis:**
 - Ein Kreis in G , der jeden **Knoten** genau einmal enthält.
- **Eulertour:**
 - Ein geschlossener Weg in G , der jede **Kante** genau einmal enthält.



Ikosaeder Spiel

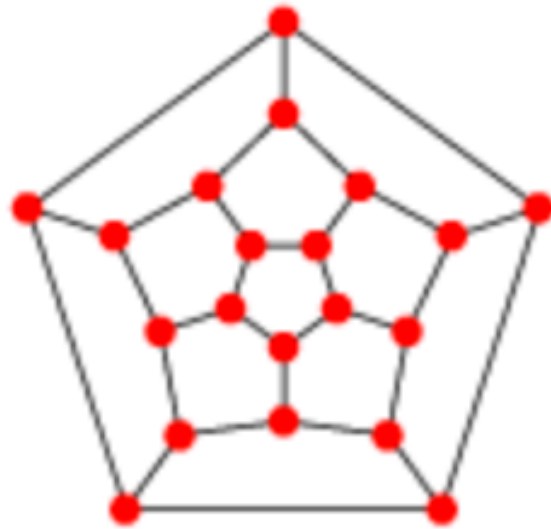


Sir William Hamilton
(1805 - 1865)

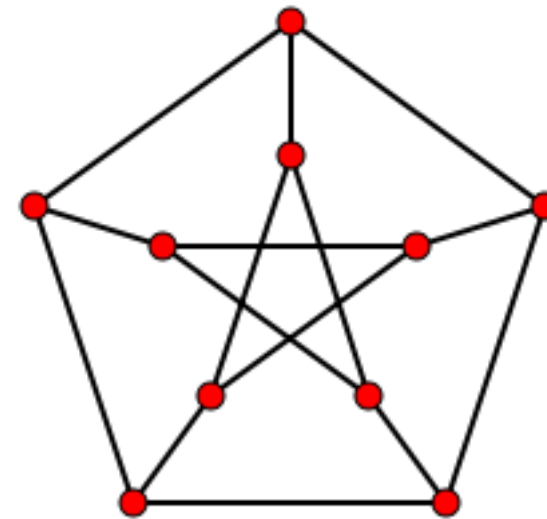
Gesucht: Hamiltonkreis



Hamiltonkreis - Beispiele

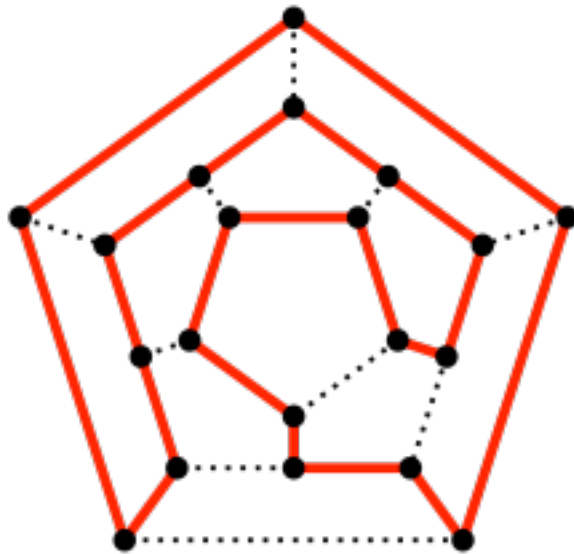


Ikosaeder

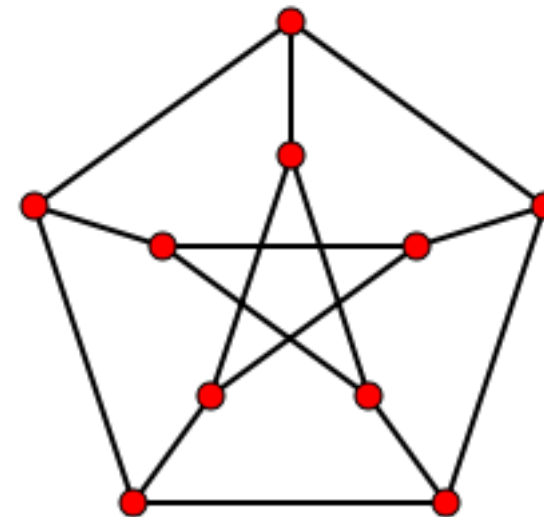


Petersengraph

Hamiltonkreis - Beispiele



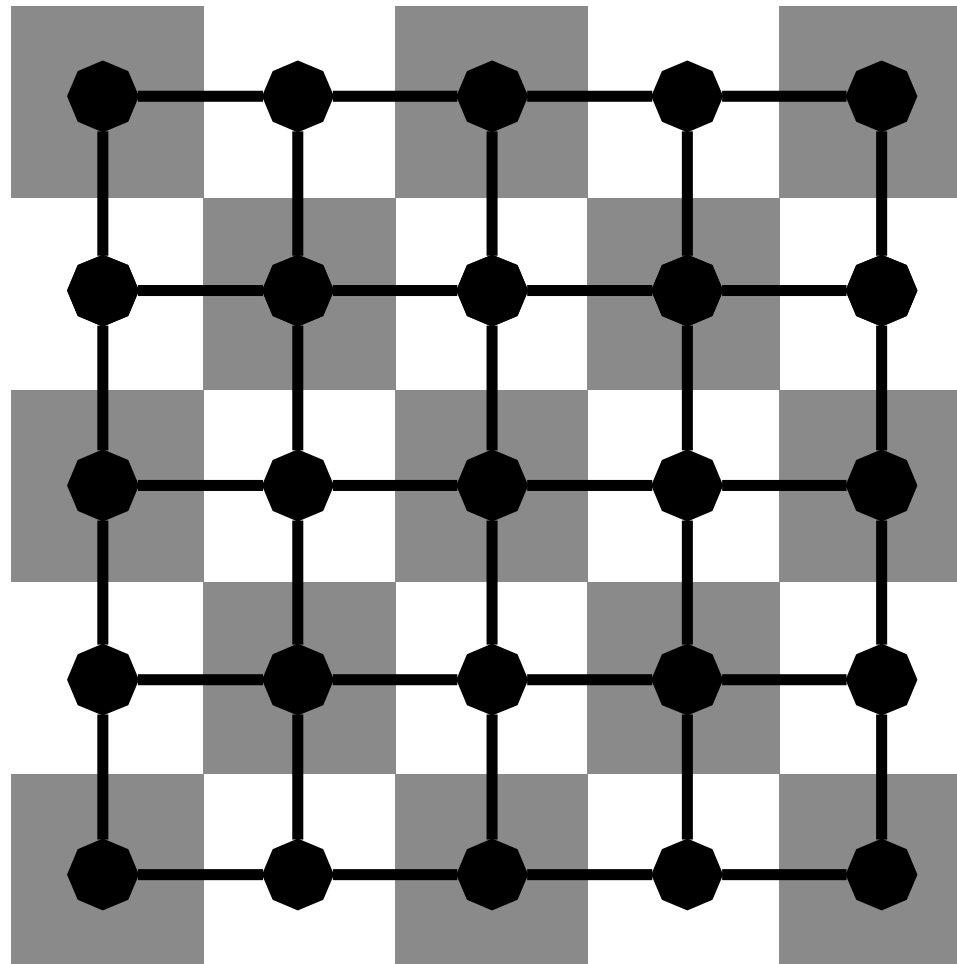
Ikosaeder



Petersengraph



Hamiltonkreis - Beispiele



Satz: Seien $m, n \geq 2$.

Ein $n \times m$ Gitter enthält einen Hamiltonkreis gdw $n \cdot m$ gerade ist.

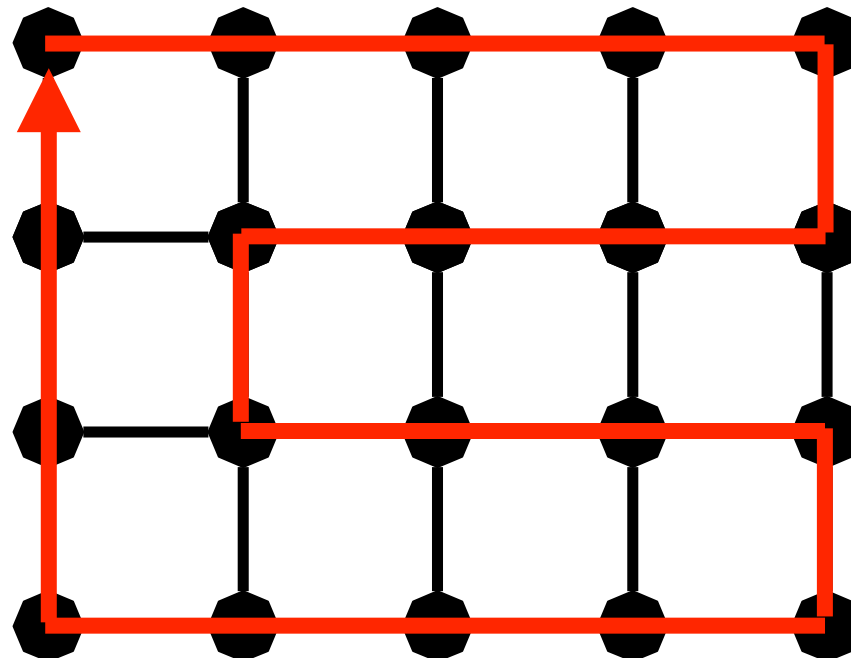
Satz: Seien $m, n \geq 2$.

Ein $n \times m$ Gitter enthält einen Hamiltonkreis gdw $n \cdot m$ gerade ist.

Beweis:

„ $\Rightarrow$ “ Schachbrett-Argument

„ $\Leftarrow$ “ siehe Skizze:



Hamiltonkreis - Beispiele



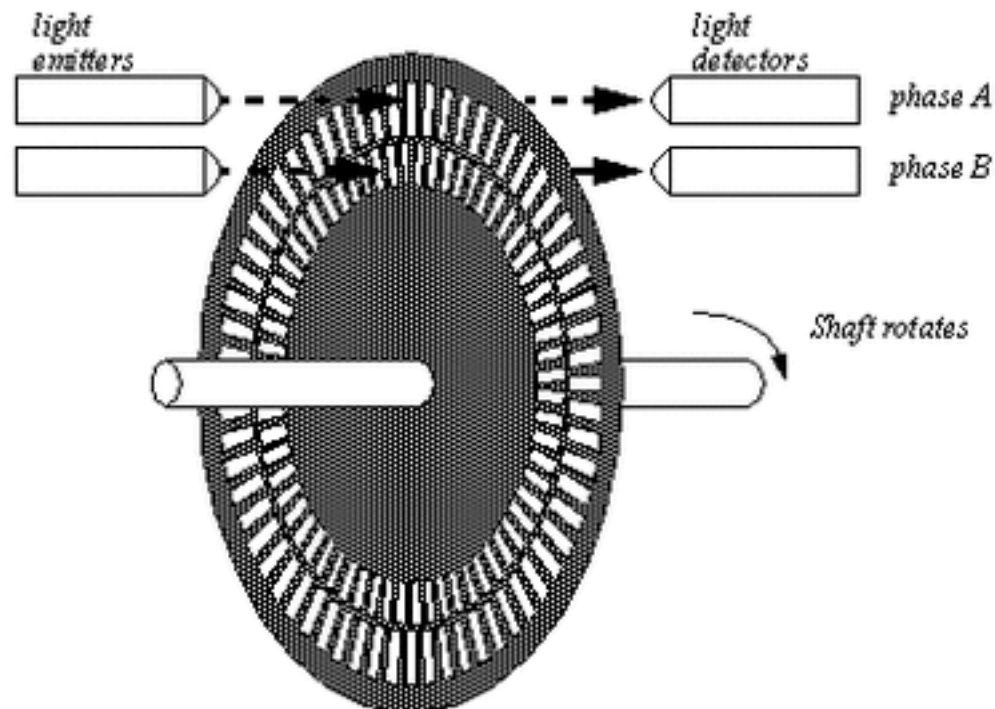
Rotary Encoder - Illuminated (RGB)

COM-10982 ROHS ✓ ⚡

★★★★☆ 8

Description: Rotary encoders can be used similarly to potentiometers. The difference being that an encoder has full rotation without limits (It just goes round and round). This is a quadrature encoder and outputs similar to 2-bit **gray code** so that you can tell how much and in which direction the encoder has been turned. They're great for navigating menu screens and things like that.

\$3.95



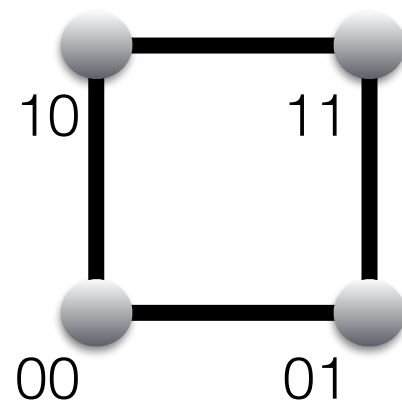
n Sensoren

d-dimensionaler Hyperwürfel H_d

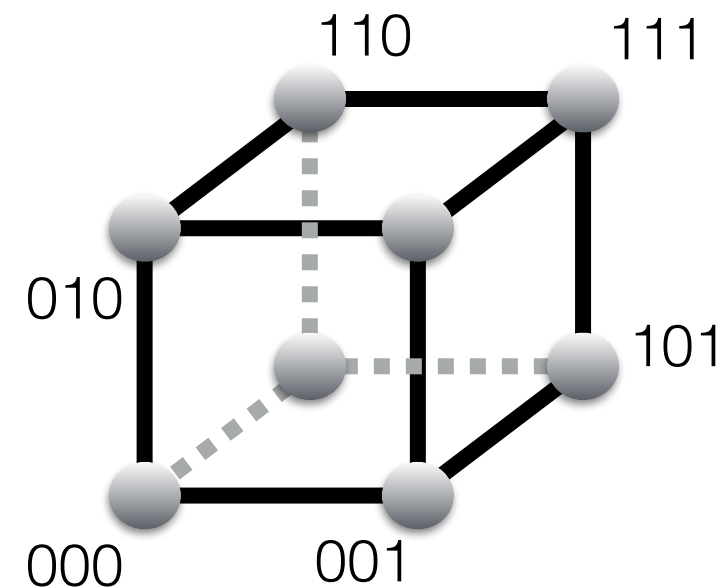
Knotenmenge: $\{0,1\}^d$

Kantenmenge: alle Knotenpaare, die sich in genau einer Koordinate unterscheiden

d=2:



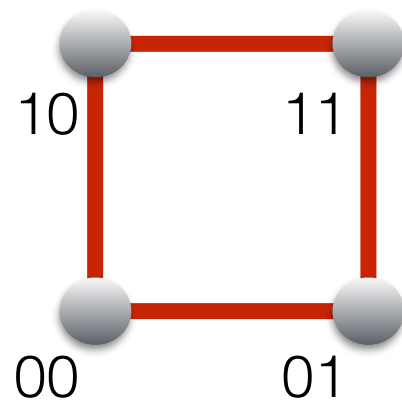
d=3:



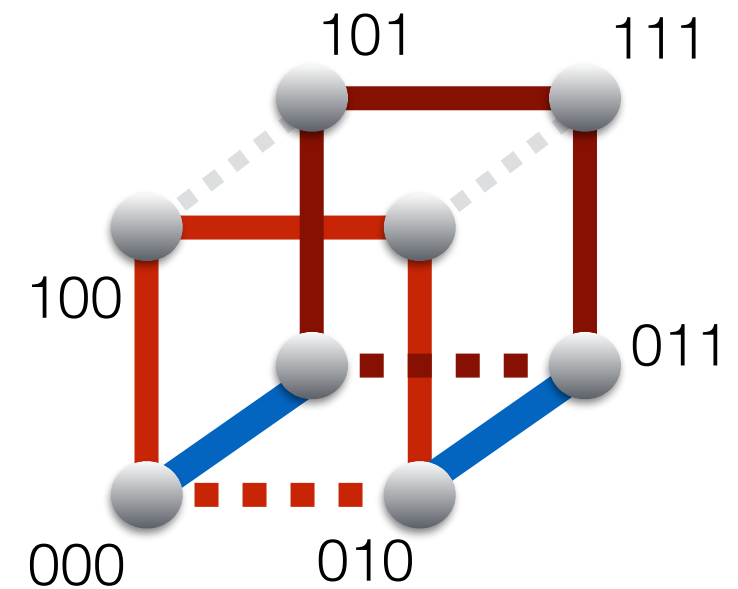
Enthält H_d einen Hamiltonkreis ?

Hamiltonkreis im Hyperwürfel: Gray-Code

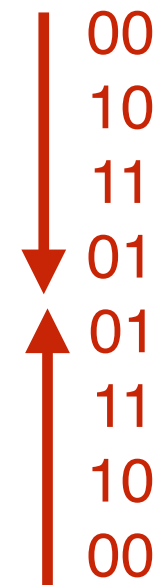
d=2:



d=3:



00
10
11
01



analog für $d \geq 4$